

Java-גרסה ממושטת (ב Verlib מול MV-RLU מסמך החלטות פישוט — השוואת

פרויקט גמר, תכנות מרובה ליבות • אורי • יולי 2026

1. המנגנונים בתמצית (מה שחייבים להבין משני המאמרים)

MV-RLU (Kim et al., ASPLOS 2019)

שאלה	המנגנון במאמר
איך נוצרת גרסה? בכתיבה?	מהחדש לישן. כותב, copy objects של (version chain) שרשרת גרסאות + master object = כל אובייקט -שלו ישן מה local-ts או אם ה (כותב אחר באמצע) p-pending נכשל אם יש: master-על ה try_lock קורא בהצלחה: עותק חדש. abort ו-retry. write-latest-version-only (כלל) של ראש השרשרת commit-ts → CAS. ב-p-pending מעגלי פר-חוט ומותקן כ log-נכתב ל
איך commit עובד?	p-הופך גלוי אטומית — ה (critical section כל העותקים מאותו) write set-כל ה: read_unlock-ב היא commit-ts נקבע מהשעון. קביעת ה write set של ה commit-ts-עובר לראש השרשרת, וה pending נקודת הלינאריוזציה
איך קורא? מוצא גרסה?	הולך על השרשרת ומחזיר את הגרסה הראשונה. dereference. מהשעון local-ts לוקח read_lock-ב master-אם אין — את ה: commit-ts ≤ local-ts עם
איך GC עובד?	concurrent (של עצמו log-כל חוט מפנה את ה. RCU בסגנון grace periods מזהה gp-detector) חוט רקע אחד (לקצר שרשראות) grace period אחרי master-הגרסה האחרונה נכתבת חזרה ל. (autonomous GC) dereference watermark (חם) ו-dereference watermark (מתמלא log-ה) capacity watermark: ומפונה אחרי שניים. שני טריגרים (גישות לעותקים גבוה
שעון	בין ליבות — כדי שהקצאת clock skew לגישור על ORDO פרימיטיב + (RDTSCP) שעון חומרה timestamps לא תהיה צוואר בקבוק
רמת עקביות	אפשר לסריאליזב ידנית ע"י נעילת אובייקטי. (write skew מתיר — serializable לא) Snapshot Isolation (ברשימה hand-over-hand למשל, try_lock_const) קריאה

Verlib (Blleloch & Wei, PPOPP 2024) — vCAS (Wei et al., PPOPP 2021) דרך מנגנון הבסיס

של אותם מחברים. המימוש שלנו מבוסס על אלגוריתם PPOPP'21 מ-versioned CAS-הוא הרחבה של מנגנון ה Verlib V1. הבסיס (שפורסם עם פסאודו-קוד מלא והוכחת לינאריוזביליות), ראו החלטה

שאלה	המנגנון
איך נוצרת גרסה בכתיבה?	$VNode = \{val, ts, nextv\}$ ל- $VHead$ מצביע: <code>VersionedCAS</code> כל מצביע בר-שינוי הוא <code>vCAS(old, new)</code> : קורא <code>head</code> , מוודא <code>val==old</code> , מקצה <code>VNode</code> חדש (<code>ts=TBD, nextv=head</code>), כמו במבנה <code>CAS</code> הוא פשוט כשל <code>CAS</code> של "טרנזקציה" — כשל <code>abort</code> אין נעילות, אין <code>VHead</code> על <code>CAS</code> -1 המקורי.
איך timestamp נקבע?	<code>camera</code> -קורא את שעון ה- <code>initTS</code> ואז <code>TBD</code> מותקן עם <code>VNode</code> ה- (helping) בדיעבד ובעזרה הדדית : עוזר לקבוע לפני שהוא ממשיך. זה (קורא, כותב שנכשל) <code>TBD</code> עם <code>head</code> כל מי שפוגש <code>CAS</code> -וקובע אותו ב- <code>timestamp</code> הטריק שהופך "הוסף לרשימה + קרא שעון + כותב לאטומי-למראה".
איך snapshot עובד?	כשל = מישור) אחד להגדיל <code>CAS</code> קורא אותו ומנסה <code>takeSnapshot</code> . מונה גלובלי <code>camera</code> אובייקט קטן/שווה. <code>ts</code> הולך על רשימת הגרסאות עד <code>readSnapshot(ts)</code> — $O(1)$. <code>retry</code> אחר הגדיל, לא צריך <code>MV-RLU</code> -לא בכתיבות — הפוך מ- <code>takeSnapshot</code> שימו לב: השעון מתקדם רק ב- <code>takeSnapshot</code> .
GC	אוסף <code>JVM</code> -של המחברים — ה <code>Java</code> בגרסת) ישנים <code>VNodes</code> לניתוק <code>epoch-based reclamation</code> (אחרי הניתוק).
רמת עקביות	<code>snapshot</code> (הוכחה מלאה במאמר) <code>snapshot</code> לינאריזביליות מלאה, כולל שאילתות.
החידוש של Verlib'24	תוך הימנעות כמעט-תמידית מהעקיפין; (2) <code>recorded-once</code> הסרת מגבלת — <code>indirection-on-need</code> (1) <code>lock-free locks</code> -לתמיכה ב- <code>idempotent CAS</code> .

ההבדל המהותי (משפט אחד לכל ציר)

- **מודל כתיבה**: `MV-RLU` = `multi-pointer update`; `vCAS` = `CAS` ו-`abort/retry` טרנזקציות נעילה עם `lock-free` בודד בלי טרנזקציות.
- **רשימת גרסאות תלויה על כל אובייקט** `vCAS` פר-חוט (מרוכז לכותב); `MV-RLU` = `log`; **מיקום הגרסאות** (מבוזר).
- **לכל אחד** `(takeSnapshot)` הצלמים `vCAS`-ב-`(commit-ב)` הכותבים `MV-RLU` מי מקדם את השעון: ב צוואר הבקבוק שלו.
- **מול לינאריזביליות** — לא זהות! (ראו סעיף 4) **עקביות** `SI`.

2. MV-RLU-Simplified — החלטות פישוט.

per-thread logs, timestamp ordering, ו-GC מבוסס `grace periods` נדחה — **משפט לדוח**: "מימשנו גרסה ממושטת הלוכדת את מנגנוני הליבה של תוך ויתור מתועד על — `grace periods` נדחה מבוסס `GC` ו-`timestamp ordering`, `thread logs` — אופטימיזציות שאינן משנות את אופי הגישה".

M1 — `commit-ב` `(getAndIncrement)` כותב מבצע **`RDTSCP+ORDO`** גלובלי במקום `(AtomicLong)` **שעון** — `read_lock-ב` `(get)` קורא `clock-skew` פותר בעיית `ORDO` ו-`TSC` אין גישה נוחה ל-`Java` ניםוק; ל-`lock` `read-ב` קורא אנחנו מחזירים בפועל את השעון הגלובלי — `timestamps` מונה אטומי יחיד. מה מאבדים: את פתרון צוואר הבקבוק של הקצאת `(H3` ראו השערה) בא להחליף. זו סטייה מהותית שחובה לציין בדוח, וגם מדידה מעניינת בפני עצמה `MV-RLU`-ש.

M2 — GC: `activeTs` (של `local-ts`) **ברקע**. כל חוט מפרסם מצב `gp-detector` קואופרטיבי במקום חוט: `capacity` של חוט חוצה את `log`-כשה (`false sharing` פר-חוט, מרופד נגד, `volatile`) "`critical section`" של כל גרסה שיש לה גרסה `log`-ומפנה מה, `minActiveTs` מחשב, `activeTs`-החוט עצמו סורק את כל ה, `watermark`, של עצמו, בלי לחסום `log`-נימוק: לוכד את הרעיון "כל חוט מפנה את ה. `commit-ts < minActiveTs` חדשה יותר עם `broadcast` קוראים", בלי תזמורת של חוט רקע ופרוטוקול `grace period`-מה מאבדים: אסינכרוניות מלאה של זיהוי ה. `gc` (אצלנו הסריקה על מסלול הכותב).

M3 — ועל המנגנון `dereference watermark`-בלבד. מוותרים על ה `capacity watermark`: יחיד GC טריגר `traversal/GC`; השני הוא כוונן עדין של איזון `watermark`-ה "אוטונומי". נימוק: ה אפשר לדווח על אורך שרשראות ממוצע `traversal/GC`; השני הוא כוונן עדין של איזון `watermark`-ה "אוטונומי". נימוק: ה בבנצ'מרק במקום לפתור את זה.

M4 — לנצח `traversal` נשמר. זה כן ליבה — בלעדיו כל קריאה משלמת `master`: של הגרסה האחרונה ל `write-back` מעתיקים, `grace period`-אם הגרסה האחרונה עומדת בתנאי ה, GC, והשוואת מסלול הקריאה מתעוותת. מפושת לכלל: בזמן וגוזמים את השרשרת `master`-אותה ל.

M5 — בין `address spaces` ידני, אין הפרדת `free` אוסף. אין `JVM`-פינוי זיכרון בפועל: ניתוק הפניות בלבד, ה (E3 ראו) הערת הוגנות: זה נכון באותה מידה לשני הצדדים (Java-שאין לה מקבילה ב C אופטימיזציה) לעותקים `masters`.

M6 — GC מלא גם אחרי `log`-אם ה. `resizing` בלי, (כניסות K למשל 64) פר-חוט: מערך מעגלי בגודל קבוע `log` — החוט נחסם עד שמתפנה מקום (גם המימוש המקורי סטטי ומועד לחסימה כזו, כמצוין במאמר).

M7 — כפי שהמאמר, `predecessor`-על ה `try_lock_const` עם `hand-over-hand`: סריאליזביליות ברשימה `vCAS` אצלנו סריאליזביליות והפער מול הלינארזביליות של `insert/remove`-עצמו מציע לרשימה מקושרת. כך פעולות ה `vCAS` אצלנו סריאליזביליות והפער מול הלינארזביליות של `insert/remove`-עצמו מציע לרשימה מקושרת. כך פעולות ה `snapshot`-מצטמצם לפעולות ה (ראו סעיף 4).

3. Verlib-Simplified — החלטות פישוט

`Verlib` — `versioned pointers` עם `camera object` ו-`Verlib` בנוי, בגרסת `Verlib` שעליו, Wei et al. (PPoPP'21) של `vCAS`-בזמן קבוע — לפי אלגוריתם ה `snapshots` מחוץ לתכולה `Verlib'24` (`indirection-on-need`, `idempotent CAS`) האינדירקציה המלאה. השיפורים של "ומצוינים כהרחבה".

V1 — `Algorithm 1` מ-PPoPP'21 כפי שהוא (`Camera`, `VNode`, `initTS`, `vCAS`, `vRead`, `readSnapshot`). הוא בדיוק (`initTS`)-`helping` נימוק: פסאודו-קוד מלא + הוכחת לינארזביליות מפורסמים; מנגנון ה. `readSnapshot` עצמו אינו זמין בגישה פתוחה, וזה `Verlib'24` של PDF-ה "נקודה העדינה" שנרצה להסביר בעל-פה, והוא בן 15 שורות. ה עם עקיפין תמידי, שהיא חסם `vCAS`-כלומר אנחנו מודדים את גרסת ה — `indirection-on-need` מתועד. מה מאבדים: את חובה לומר זאת ליד כל מסקנה על מסלול הקריאה. `Verlib` עליון על עלות הקריאה של גישה.

V2 — לרשימה. המאמר מראה שברשימה `recorded-once` אופטימיזציה (אופציונלי, רק אם יום 2 מסתיים מוקדם) עצמו ולחסוך את העקיפין. אם ממומש — מקבלים נקודת `node`-בתוך ה `ts+nextv` מקושרת חד-כיוונית אפשר להטמיע את אמיתי", וזה משרדג את סיפור ההשוואה. לא מתחייבים `Verlib` "מלא" ל `vCAS` בנינים בין.

V3 — אותו מנגנון) פעילים `handles` פעיל מינימלי. שומרים רישום של `snapshot` של גרסאות: גיזום עצל לפי GC `publish` פעולות, כותב שמוסיף גרסה גוזם מהזנב של רשימת הגרסאות של האובייקט N מדי. (סימטריה מכוונת — M2 כמו `epoch-based` אוסף. נימוק: מחליף `JVM`-ה. `ts < minActiveSnapshot` שיש חדש ממנו עם `VNode` הספציפי כל `reclamation` מלא במנגנון של 20 שורות עם אותה סמנטיקת בטיחות.

מעוברסנים; שדות next **אחת גלובלית לכל מבנה הנתונים** (כמו במימושי המחברים), רק מצביעי **V4 — camera** נשארים רגילים — זו אופטימיזציה לגיטימית מהמאמר עצמו (סעיף 5 שם) immutable (key)

4. הערות הוגנות השוואה (לשמור לדות, סעיף מתודולוגיה)

כל כתיבה מגדילה את המונה הגלובלי; ב MV-RLU-S-א-סימטריית השעון היא תכונה, לא באג של ההשוואה. ב E1 וציר "אחוז כתיבות" צפויים להזיז את שתי הגישות בכיוונים "snapshots מגדיל אותו. לכן ציר "אחוז *snapshot* כל vCAS M1 הפוכים — זה בדיוק סוג נקודת החיתוך שאנחנו מחפשים, אבל צריך לזכור שחלק מהאפקט אצלנו מוגבר בגלל

E2 — אצלנו: פעולות רשימה סריאליזביליות) SI נותן MV-RLU; לינאריזבילי CAS והגישות לא מספקות אותה ערובה. **versioning** ההשוואה תקפה ברמת "גישות. (סמנטיקה SI עדיין MV-RLU-S של snapshot אבל שאילתת, M7 בזכות נקנה בערובה חלשה יותר MV-RLU-S של throughput-למבני נתונים", ובדוח מציינים שה

E3 — האמיתית נבלעת GC-שוניות. בשני הצדדים "פינוי" = ניתוק הפניות; עלות ה **reclamation** **מסתר עלויות JVM-ה — E3** מ-GC time ומדווחים גם (-Xms=-Xmx) גדול וקבוע heap ועלולה להיזקף לחוסים אחרים. מריצים עם JVM-ב-
JFR/verbose:gc. אם התוצאות השודות

E4 — seeds של מצב חוטים, אותם publish בסיסי, אותו מנגנון Node layout אותו, harness תשתית זהה: אותו. העצמו versioning-ההבדל היחיד בין שתי המערכות הוא מנגנון ה

ממשק משותף (לקבע ביום 1 לפני כל קוד).

```
java

interface VersionedSet {
    boolean insert(int key);
    boolean remove(int key);
    boolean contains(int key); // קריאה "עכשווית" - מסלול מהיר
    long sumRange(int lo, int hi); // version-כאן ה - nodes-מרבובת snapshot פעולת
    int sizeSnapshot(); // snapshot הכול פעולת
}
```

— ואם יש זמן, `VcasList`, `MvRluList`, (לבריקות baseline + oracle) `CoarseLockList` מימושים
`MvRluBst` / `VcasBst`.

השערות מדידה (לנסח לפני. לבדוק אחרי – יום 4 נשען על זה) 6.

- **H1 (snapshots קריאה-כבד, בלי):** MV-RLU-S master מעט מהיר — קורא פוגש write-
back ישירות אחרי master מעט מהיר — קורא פוגש write-back ממומש, הפער V2 על כל מצביע. אם (VHead→VNode) משלם עקיפין אחד vCAS בעוד, (אפס עקיפין)
אמור להיסגר.
- **H2 (כתיבה-כבד):** vCAS מכותבי — KVM-RLU-S גם משלמים גם KVM-RLU-S מהיר יותר אצלנו — מכותבי vCAS
על השעון הגלובלי increment משלמים גם KVM-RLU-S מהיר יותר אצלנו — מכותבי vCAS
בלי לגעת בשעון. חשוב: חלק מהפער CAS + משלמים הקצאה vCAS מכותבי; WW על log גם, גם (M1)

לציין — M1 הוא ארטיפקט של

- **H3 (snapshots ותכופים):** ה: camera-היפוך של vCAS הנקודת הפכת הcontention (המאמר) רק קוראים את השעון MV-RLU-S בעוד קוראי, (בדיוק מסיבה זו succ1 עצמו מדד 37-41% תקורה על
- **H4 (snapshots לכתובות):** readSnapshot עלות ה-CAS-פרופורציונלית למספר ה snapshot (פר-אובייקט) GC-אורך השרשרת תלוי בקצב ה-MV-RLU-S ב; snapshot-יחס כתיבות $\leftrightarrow$ אורך ה
- **H5 (סקיילינג בחוטים):** שתי הגישות מוגבלות אצלנו ע"י מונה גלובלי כלשהו; מי שהעומס מפעיל את המונה שלה פחות — מנצחת בקצה החוטים הגבוה

7. מה נשלח למשה היום (שתי שורות)

RLU-MV-b) אני מממש גרסאות מפושטות שלוכדות את מגננוני הליבה: scope: היי משה, התחלתי. שתי הבהרות" multi-versioning + per-thread logs + GC עם AtomicLong נדחה, Verlib (indirection-on-need) שעליו הספרייה בנויה, בלי PPoPP'21-m-CAS versioned מגננון ה: Verlib מבנה. כל ההחלטות מתועדות snapshot; BST נתונים ראשי: רשימה מקושרת ממוינת עם פעולות כהרחבה אם יספיק הזמן. תקיו מבחינתך?"